

Using OpenLDAP with TIBCO Enterprise Message Service™

This whitepaper is intended to provide a high-level look at how TIBCO Enterprise Message Service™ uses Lightweight Directory Access Protocol (LDAP) for Authentication of Users, and by storing Group Membership, how it aids in Authorization.

This document is not exhaustive – it is intended to give the user the ability to do a simple Authorization and Authentication “proof-of-concept” using LDAP.

We will be using the following products:

OpenLDAP™ for Windows 2.2.9 (freely distributable)

OpenSSL 0.9.7g (freely distributable)

Softerra LDAP Browser 2.6 (freely distributable)

and TIBCO Enterprise Message Service 4.2™

Version 1.3
20-Sept-2006



<http://www.tibco.com>

Global Headquarters

3303 Hillview Avenue
Palo Alto, CA 94304
Tel: +1 650-846-1000
Toll Free: 1 800-420-8450
Fax: +1 650-846-1005

©Copyright 2005, TIBCO Software Inc. All rights reserved. TIBCO, the TIBCO logo, The Power of Now, and TIBCO Software are trademarks or registered trademarks of TIBCO Software Inc. in the United States and/or other countries. All other product and company names and marks mentioned in this document are the property of their respective owners and are mentioned for identification purposes only. 0204

Copyright Notice

COPYRIGHT© 2005, 2006 TIBCO Software Inc. This document is unpublished and the foregoing notice is affixed to protect TIBCO Software Inc. in the event of inadvertent publication. All rights reserved. No part of this document may be reproduced in any form, including photocopying or transmission electronically to any computer, without prior written consent of TIBCO Software Inc. The information contained in this document is confidential and proprietary to TIBCO Software Inc. and may not be used or disclosed except as expressly authorized in writing by TIBCO Software Inc. Copyright protection includes material generated from our software programs displayed on the screen, such as icons, screen displays, and the like.

Trademarks

Technologies described herein are either covered by existing patents or patent applications are in progress. All brand and product names are trademarks or registered trademarks of their respective holders and are hereby acknowledged.

Confidentiality

The information in this document is subject to change without notice. This document contains information that is confidential and proprietary to TIBCO Software Inc. and may not be copied, published, or disclosed to others, or used for any purposes other than review, without written authorization of an officer of TIBCO Software Inc. Submission of this document does not represent a commitment to implement any portion of this specification in the products of the submitters.

Content Warranty

The information in this document is subject to change without notice. THIS DOCUMENT IS PROVIDED "AS IS" AND TIBCO MAKES NO WARRANTY, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO ALL WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. TIBCO Software Inc. shall not be liable for errors contained herein or for incidental or consequential damages in connection with the furnishing, performance or use of this material.

For more information, please contact:

TIBCO Software Inc.
3303 Hillview Avenue
Palo Alto, CA 94304
USA

Table of Contents

Overview.....	5
What is LDAP? What is OpenLDAP?.....	5
How Does TIBCO Enterprise Message Service use LDAP?	6
Installing OpenLDAP	8
Configuring and Testing OpenLDAP	9
Configuring SLAPD.CONF	9
- Including LDAP Schemata	9
- Adding a Suffix and Root Distinguished Name (DN)	10
Creating and Editing LDAP Data Interchange Format Files (LDIF)	11
- Creating a ROOT Node.....	11
- Adding a Root Node	12
Adding Users and Groups	12
- Create and Edit: people.ldif.....	12
- Add people.ldif.....	12
- Create and Edit: groups.ldif.....	12
- Add groups.ldif	12
- Create and Edit: testperson.ldif.....	13
- Add testperson.ldif.....	13
- Create and Edit addmember.ldif	14
Starting OpenLDAP	14
Testing OpenLDAP	15
Configuring TIBCO Enterprise Message Service for LDAP	17
Edit tibemsd.conf	17
Debugging EMS	18
Testing Authentication.....	18
Java Programs	18
BusinessWorks.....	21
LDAP/S – securing the connection between EMS and LDAP	21
Create or find appropriate X.509v3 Certificates	22
Re-edit slapd.conf.....	22
Re-edit tibemsd.conf.....	23
New Command-line for Starting OpenLDAP	23
New Procedures for Start-Up	24
Listening on the Port for LDAP/S - 636	24
Testing Authorization.....	24
Create Queue	25
Grant Partial Permissions.....	25
First Test.....	25
We forgot the SECURE Property!	25
Modification – adding UID & using UID rather than CN.....	26
addUID.ldif	26



LDAPModify Command 27

Modify tibemsd.conf to use the UID attribute 27

Overview

This whitepaper will guide you through the installation and implementation of an OpenLDAP™ Server to support Authentication and Authorization from the LDAP Client resident in TIBCO Enterprise Message Service™.

What is LDAP? What is OpenLDAP?

LDAP stands for Lightweight Directory Access Protocol, with Version 3 based on IETF RFC 2256 and 3377 – still in “Proposed Standard” status after almost 8 years – with RFC 3377 being an update as recently as 2002.

See: <http://www.ietf.org/rfc/rfc3377.txt?number=3377>

OpenLDAP is an Open Source implementation of LDAP, currently supporting Version 3 (RFC 3377), and it includes:

- SLAPD – stand-alone LDAP Server
- SLURPD – stand-alone LDAP Replication Server
- SDK – software development kit
- Utilities, tools, and sample clients

OpenLDAP currently does NOT support the following LDAPv3 features:

- DIT Structure Rules
- Name Forms
- Schema Updates (using LDAP)

Nor does it support the following Standard Track Extensions:

- LCUP (RFC 3928 - Use LDAPsync instead)
- Collective Attributes (RFC 3671)
- Server Side Sorting of Search (RFC 2891)
- Dynamic Directory Services (RFC 2589)

A freely distributable binary for the Windows platform is available from Lucas Bergman Software:

<http://lucas.bergmans.us/hacks/openldap/>

OpenLDAP Software can be found at:

<http://www.openldap.org/>

How Does TIBCO Enterprise Message Service use LDAP?

TIBCO Enterprise Message Service (EMS) can use LDAP to store user and group information for Authentication and Authorization. TIBCO EMS acts as an LDAP Client and retrieves ID/Password information about users and group memberships of those users. Access Control Lists are maintained in EMS proper, with LDAP access providing pointers.

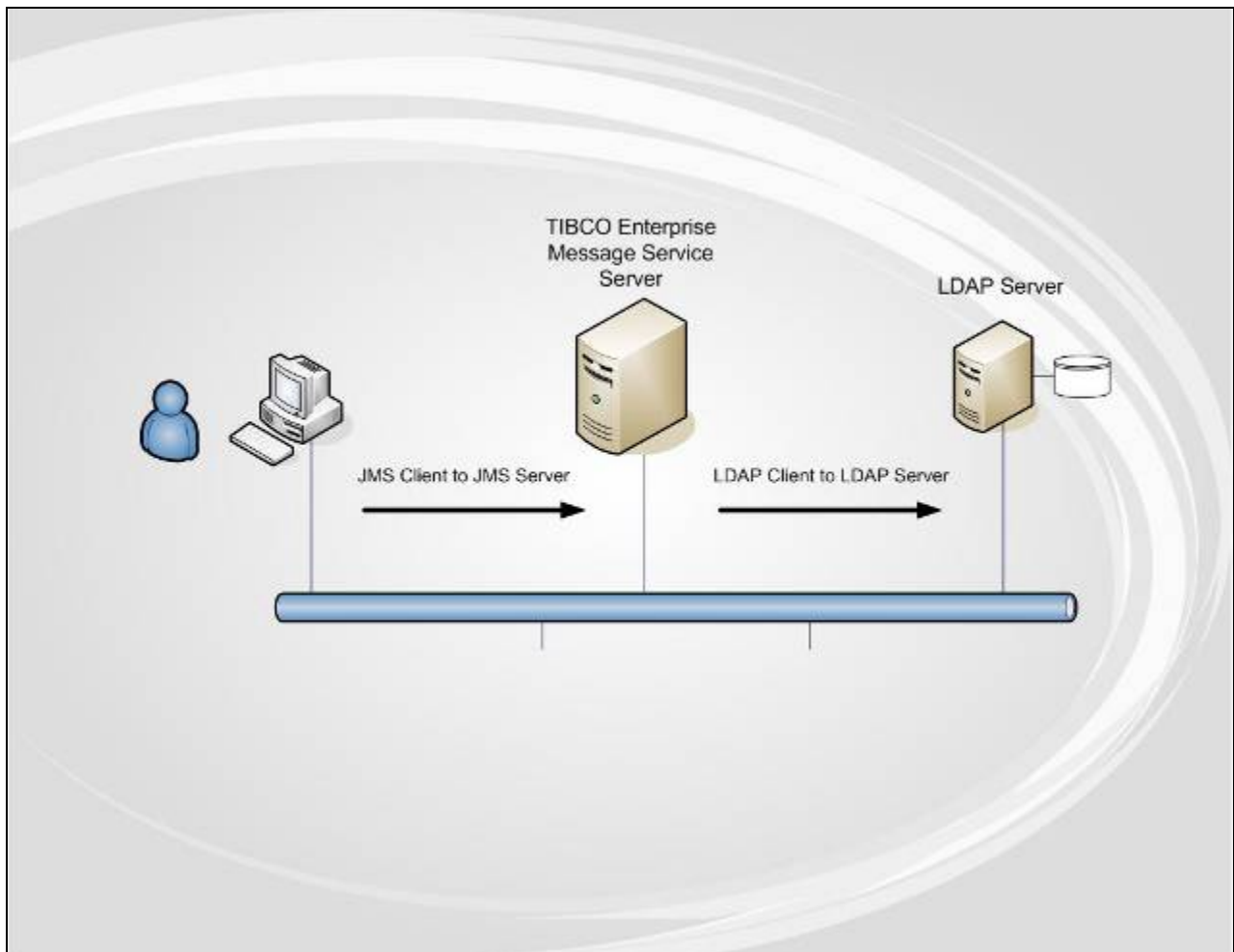


Figure 1 - Client/Server Relationships

In the figure above, a JMS Client application would present User ID (userName) and Password information, which it would pass to the JMS Provider. With the appropriate configuration, that Provider would parse the JMS Message and relay an LDAP request to the LDAP Server, retrieving group information at the same time. User ID and Password can be different for JNDI lookup and Connection Factory use.

Failure to have proper credentials results in throwing a `JMSSecurityException`.

Here is an example from the javadoc for QueueConnectionFactory:

QueueConnectionFactory (TIBCO Enterprise Message Service - 4.2.0) - Microsoft Internet Explorer

File Edit View Favorites Tools Help

Address <C:\tibco\ems\doc\html\api\Java\index.html> Go Links Google

Search Web ...attempting to retrieve buttons from Yahoo!...

TIBCO Enterprise Message Service Version 4.2.0
[All Classes](#)

Packages
[com.tibco.tibjms](#)
[com.tibco.tibjms.admin](#)

Permissions
[PrincipalInfo](#)
[ProducerInfo](#)
[Queue](#)
[QueueBrowser](#)
[QueueConnection](#)
[QueueConnectionFactory](#)
[QueueInfo](#)
[QueueReceiver](#)
[QueueRequestor](#)
[QueueSender](#)
[QueueSession](#)
[RVCMTransportInfo](#)
[RVTransportInfo](#)
[ResourceAllocationException](#)
[RouteInfo](#)
[RouteSelector](#)
[SSLParams](#)
[SSTransportInfo](#)
[ServerInfo](#)
[ServerSession](#)
[ServerSessionPool](#)
[Session](#)
[StatData](#)

Throws:
[JMSEException](#) - if the JMS provider fails to create the queue connection due to some internal error.
[JMSSecurityException](#) - if client authentication fails due to an invalid user name or password.

createQueueConnection

```
public QueueConnection createQueueConnection(java.lang.String userName,
                                             java.lang.String password)
                                             throws JMSEException
```

Creates a queue connection with the specified user identity. The connection is created in stopped mode. No messages will be delivered until the Connection.start method is explicitly called.

Parameters:
 userName - the caller's user name
 password - the caller's password

Returns:
 a newly created queue connection

Throws:
[JMSEException](#) - if the JMS provider fails to create the queue connection due to some internal error.
[JMSSecurityException](#) - if client authentication fails due to an invalid user name or password.

Package **Class** Tree Deprecated Index Help

[PREV CLASS](#) [NEXT CLASS](#) [FRAMES](#) [NO FRAMES](#)
 SUMMARY: NESTED | FIELD | CONSTR | [METHOD](#) DETAIL: FIELD | CONSTR | [METHOD](#)

TIBCO Enterprise Message Service Version 4.2.0

My Computer

Installing OpenLDAP

Below are the installation screens for OpenLDAP 2.2.19-1 for Windows – it is a simple installation process...

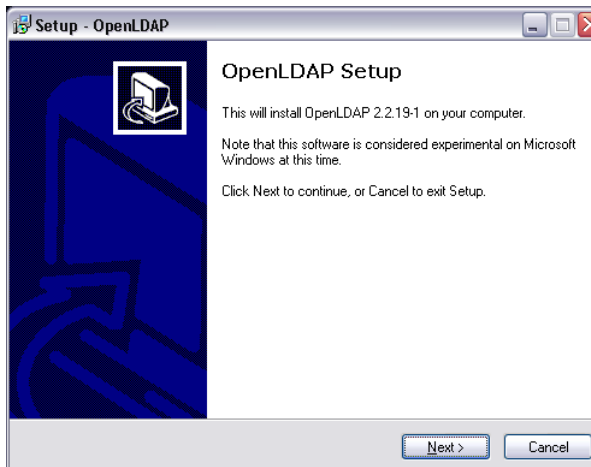


Figure 2 - First Screen for OpenLDAP Install

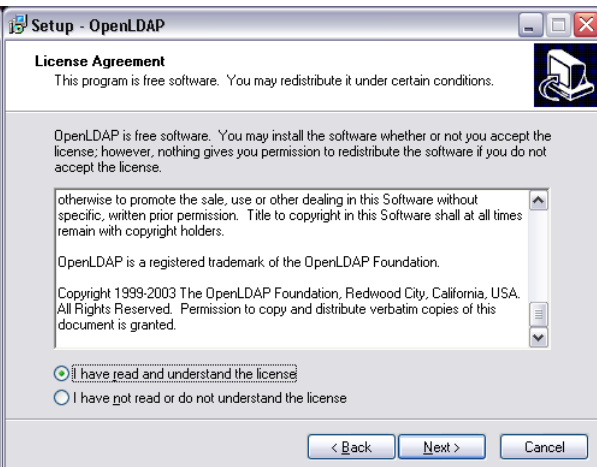


Figure 3 - Second Screen – License Agreement

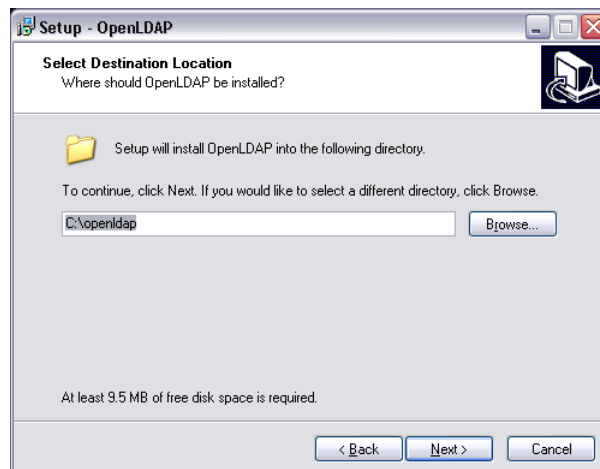


Figure 4 - Pick a Directory for Install

That was simple, wasn't it?

Configuring and Testing OpenLDAP

Configuring SLAPD.CONF

At its most basic, you start OpenLDAP by executing `slapd.exe`, as shown below, but we want some very specific attributes for use with EMS, so we will edit the main configuration file – `slapd.conf`.



```

me $ ref $ vendorVersion $ vendorName $ supportedSASLMechanisms > >
  2.5.13.4 <caseIgnoreSubstringsMatch>: matchingRuleUse: < 2.5.13.4 NAME 'case
IgnoreSubstringsMatch' APPLIES < dnQualifier $ destinationIndicator $ serialNum
ber > >
  2.5.13.3 <caseIgnoreOrderingMatch>: matchingRuleUse: < 2.5.13.3 NAME 'caseIg
noreOrderingMatch' APPLIES < dnQualifier $ destinationIndicator $ serialNumber >
>
  2.5.13.2 <caseIgnoreMatch>: matchingRuleUse: < 2.5.13.2 NAME 'caseIgnoreMatc
h' APPLIES < uid $ dmdName $ houseIdentifier $ dnQualifier $ generationQualifier
$ initials $ givenName $ destinationIndicator $ physicalDeliveryOfficeName $ po
stOfficeBox $ postalCode $ businessCategory $ description $ title $ ou $ o $ str
eet $ st $ l $ c $ serialNumber $ sn $ knowledgeInformation $ labeledURI $ cn $
name $ ref $ vendorVersion $ vendorName $ supportedSASLMechanisms > >
  2.5.13.1 <distinguishedNameMatch>: matchingRuleUse: < 2.5.13.1 NAME 'disting
uishedNameMatch' APPLIES < seeAlso $ roleOccupant $ owner $ member $ distinguish
edName $ aliasedObjectName $ namingContexts $ subschemaSubentry $ modifiersName
$ creatorsName > >
  2.5.13.0 <objectIdentifierMatch>: matchingRuleUse: < 2.5.13.0 NAME 'objectId
entifierMatch' APPLIES < supportedApplicationContext $ supportedFeatures $ suppo
rtedExtension $ supportedControl > >
slapd.exe startup: initiated.
backend_startup: starting "dc=my-domain,dc=com"
bdb_db_open: dbenv_open(C:/openldap/var/openldap-data)
slapd starting
  
```

- Including LDAP Schemata

There are some very basic schemata that you will be using:

- `core.schema` – OpenLDAP’s required fundamental schema – this is based on RFCs 2251-2256
- `cosine.schema` – related to X.500 – based on RFC 1274
- `inetorgperson.schema` – the basic “person” schema – RFC 2798
- `java.schema` – as defined in RFC 2713, it is used to store serialized java objects – like for JNDI. NOTE: EMS doesn’t use LDAP for JNDI, but your clients can!
- `openldap.schema` – miscellaneous stuff, housekeeping etc.

Start up your favorite text editor and add references to these schemata as shown below – note that this is only a snippet of the `slapd.conf` file (additions shown in bold):

```
#
# See slapd.conf(5) for details on configuration options.
# This file should NOT be world readable.
# This is the default
ucdata-path    C:/openldap/ucdata
include        C:/openldap/etc/schema/core.schema

# Add this as being very useful - Carlo Milono
include        C:/openldap/etc/schema/cosine.schema
include        C:/openldap/etc/schema/inetorgperson.schema
include        C:/openldap/etc/schema/java.schema
include        C:/openldap/etc/schema/openldap.schema
```

- Adding a Suffix and Root Distinguished Name (DN)

Next, we pick a backend database – BDB is the default Berkeley Data Base for this distribution, though you can use the GNU Database manager. The suffix should be familiar to Internet users as it is the Domain Component of a Fully-Qualified Domain Name (FQDN), and it is composed to two attributes, `dc=tibco` (or your organization) and `dc=com` (or your “top level domain”) – so LDAP maps quite well to Domain Naming Service.

As the LDAP classes must communicate to a back-end database, we give certain privileges to an ID – in this case we specify a rootdn (DN refers to a Distinguished Name), to which we place the suffix: `cn=root,dc=tibco,dc=com`

We also supply a password, which is shown in clear text as “`secret`”. You can choose other methods of storing the password, including:

- SHA or SSHA – Secured Hash (FIPS 160-1 or FIPS 160-1 with seed)
- MD5 or SMD5 – Message Digest Hash (RFC 1321 or same with seed)
- CRYPT – uses the `crypt(3)` function
- CLEARTXT – in-the-open

You would specify the type of hashing with the `password-hash` parameter, and you would generate the password with the `slappasswd` command:

```
C:\openldap>slappasswd -h {SSHA} -s secret
{SSHA}yCcWuXaTMqRcWB22Yuw68sP0G9wWB1Xl
```

```
#####
# BDB database definitions
#####

database      bdb
suffix        "dc=tibco,dc=com"
rootdn        "cn=root,dc=tibco,dc=com"
# Cleartext passwords, especially for the rootdn, should
# be avoid. See slappasswd(8) and slapd.conf(5) for details.
# Use of strong authentication encouraged.
rootpw        secret
# The database directory MUST exist prior to running slapd AND
# should only be accessible by the slapd and slap tools.
# Mode 700 recommended.
directory     C:/openldap/var/openldap-data
# Indices to maintain
index objectClass eq
```

Later, we will add some parameters to enable SSL communication from LDAP Clients (TIBCO Enterprise Message Service) to our OpenLDAP Server.

Creating and Editing LDAP Data Interchange Format Files (LDIF)

As we have told the daemon to have certain configurations, those configurations need to be instantiated within the backend database, so we will use LDAP Interchange Format files as defined in RFC 2849. We will start with building the root node of what will become our directory tree.

- Creating a ROOT Node

Using your text editor, create a file (I've called it `init.ldif`) with the following content:

```
dn: dc=tibco, dc=com
objectclass: top
objectclass: dcObject
objectclass: organization
o: tibco
dc: tibco

dn: cn=root, dc=tibco, dc=com
objectclass: organizationalRole
cn: root
```

The format is hierarchical and processed in order, so we start with the Distinguished Name of our Domain Component, which is described as belonging at the `top`, with a Domain Component object with further refinement as being an `organization` class. The Organization (`o`) is

`tibco`, and the core Domain Component is `tibco`. Next, we build the Distinguished Name for `root`, associated with the class of `organizationalRole` and give it a Common Name – `root`.

- Adding a Root Node

Next, you need to add this configuration to the server with the `slapadd` command:

```
slapadd -f slapd.conf -l init.ldif
```

Adding Users and Groups

Now that the root has been established, we want to build out the tree further by adding a branch for people and a branch for groups.

- Create and Edit: `people.ldif`

The following text creates a Distinguished Name associated with an Organizational Unit (ou) that is bound to the Domain Component. This structure will be used to store user information for Authentication:

```
dn:      ou=people, dc=tibco, dc=com
objectClass:  top
objectClass:  organizationalUnit
ou:      people
```

- Add `people.ldif`

```
slapadd -f slapd.conf -l people.ldif
```

- Create and Edit: `groups.ldif`

The next file adds the second branch, and will be used to associate users to groups, enabling Authorization.

```
dn:      ou=groups, dc=tibco, dc=com
objectClass:  top
objectClass:  organizationalUnit
ou:      groups
```

- Add `groups.ldif`

```
slapadd -f slapd.conf -l groups.ldif
```

- Create and Edit: testperson.ldif

We continue by adding our first user – this time, the objectclasses are associated with the top objectClass `person`, and we refine this further by specifying an `inetOrgPerson` schema under the `organizationalPerson` model – this represents the hierarchy of the `inetOrgPerson` object class. Remember that the LDIF is processed in order! Some new acronyms include `gn` for Given Name and `sn` for SurName.

This entry is the EMS Admin equivalent of this command:
create user test password=test

```
dn:      cn=test, ou=people, dc=tibco, dc=com
objectClass: top
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
cn: test
gn: test
sn: test
mail: test@tibco.com
physicalDeliveryOfficeName: TIBCO Software Inc.
postalAddress: 3307 Hillview Avenue
l: Palo alto
st: CA
postalCode: 94304
telephoneNumber: 650-846-5555
facsimileTelephoneNumber: 650-846-5556
pager: 650-846-5557
mobile: 650-846-5558
homePhone: 650-846-5559
userPassword: test
ou: people
```

- Add testperson.ldif

```
slapadd -f slapd.conf -l testperson.ldif
```

- Create and Edit addmember.ldif

Just as we created an Organizational Unit (ou) to hold people – and added a person – so we shall add a group and associate membership to that group.

This entry is the EMS Admin equivalent of these two commands:

create group sales

add member sales test

```
dn:      cn=sales, ou=groups, dc=tibco, dc=com
cn: sales
objectClass: groupOfNames
member: cn=test,ou=people,dc=tibco,dc=com
```

If you wish, you can use the previous LDIF files to create additional users and add them to other groups.

Starting OpenLDAP

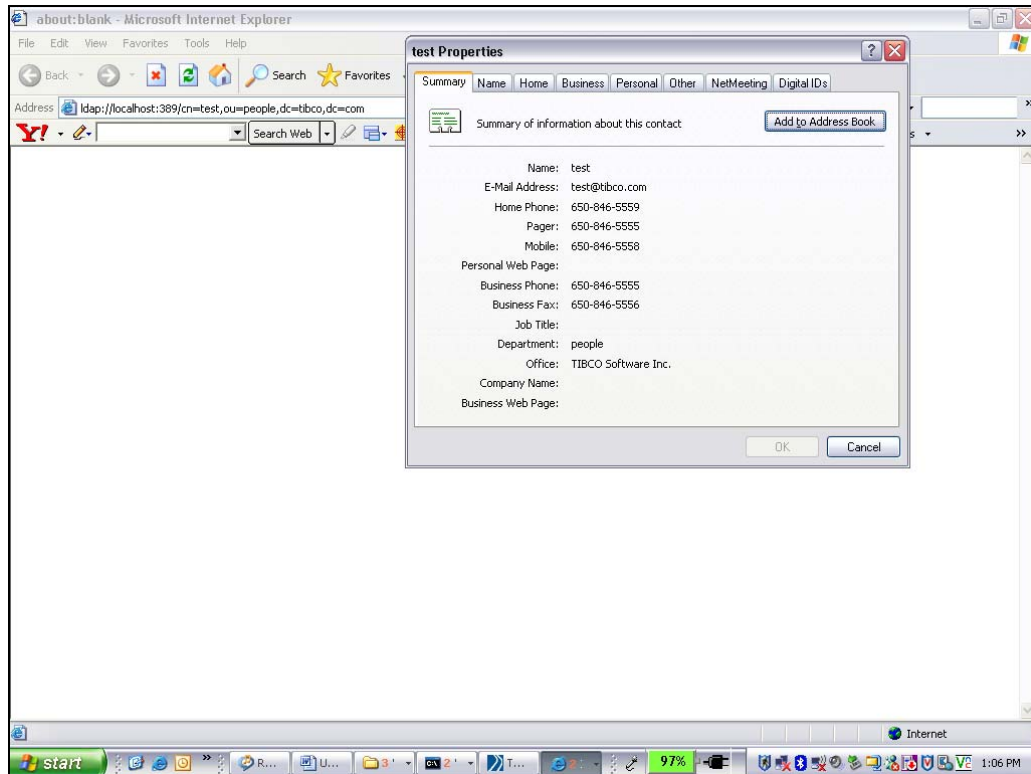
Now that we have the basic structures in place, and those structures have some form of data in them, we can start OpenLDAP. There are many flags, but we will do a simple start at this time. Open up a command prompt and move to the OpenLDAP directory and enter this command:

```
slapd -d 1
```

Testing OpenLDAP

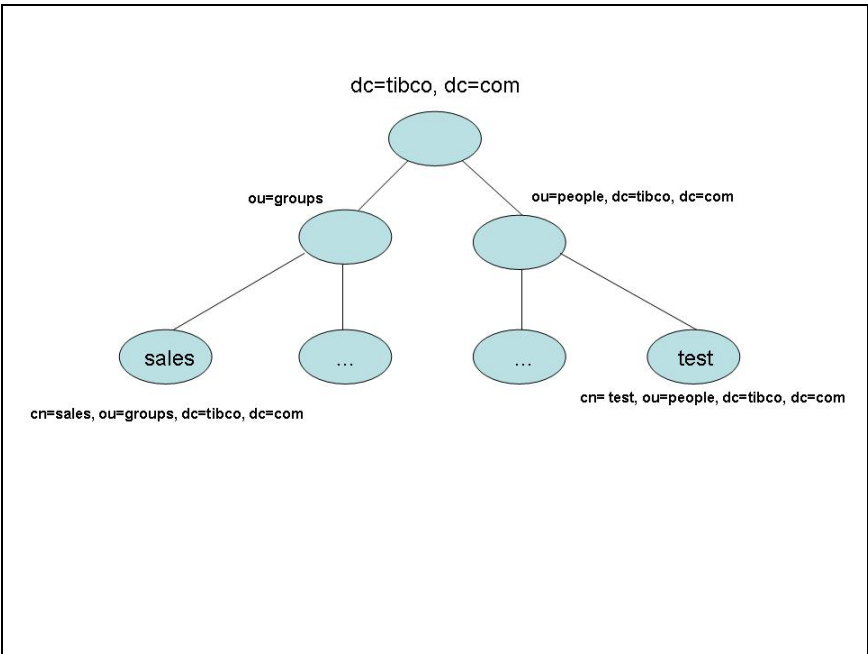
A simple test is to use an LDAP URL in your browser – so let's look for the “test” person!

`ldap://localhost:389/cn=test,ou=people,dc=tibco,dc=com`

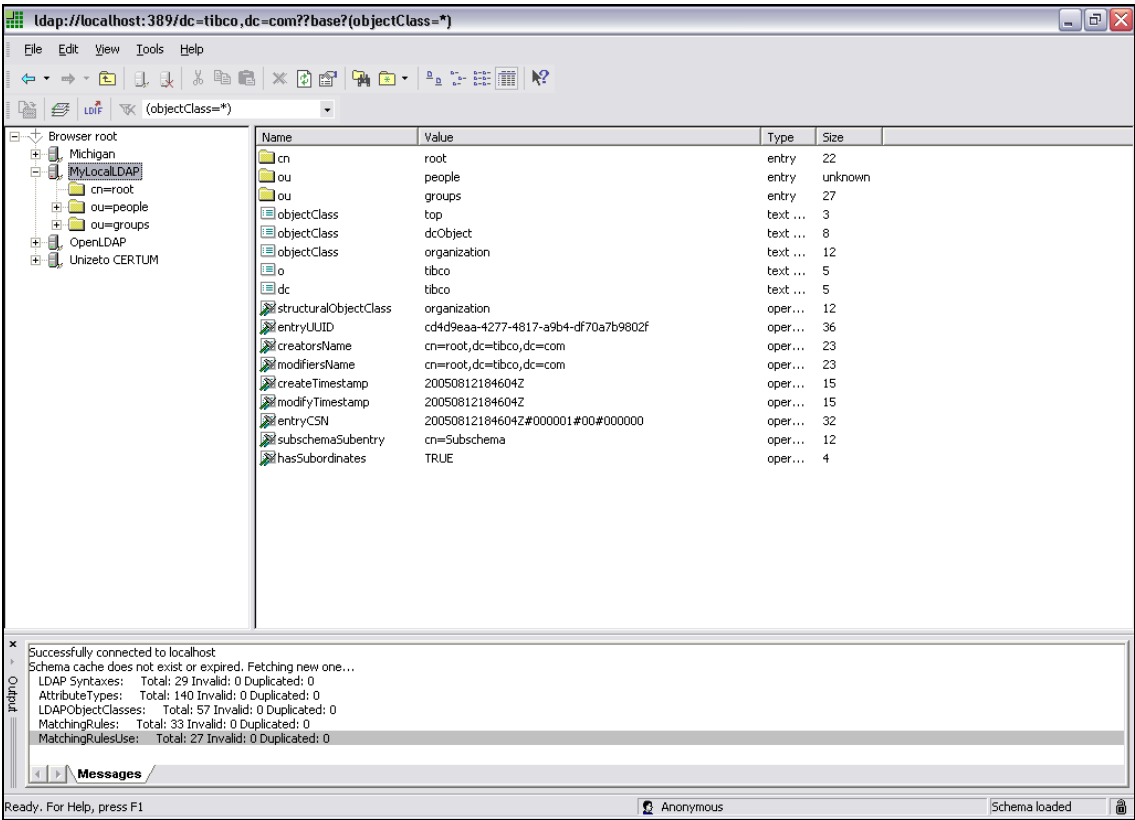


Notice that we used the fully-qualified Distinguished Name for “test”, including which Organizational Unit to search and the domain information.

Schematically, our tree looks like this:



Using a tool like Softerra LDAP Browser 2.6, we get a classic UI tree:



Configuring TIBCO Enterprise Message Service for LDAP

Edit tibemspd.conf

For the purposes of this exercise, I am blocking any other form of authorization with these parameters:

```
authorization    = enabled
user_auth        = LDAP
```

This combination enforces authorization and only through LDAP mechanisms.

Next, we enter the LDAP information:

```
##### Sample LDAP Configuration for OpenLDAP -
#           Edited by Carlo Milono
ldap_url    = ldap://localhost:389
ldap_principal    = "cn=root,dc=tibco,dc=com"
ldap_credential    = secret
ldap_user_class    = person
ldap_user_attribute    = cn
ldap_user_base_dn    = "ou=people,dc=tibco,dc=com"
ldap_user_scope      = subtree
ldap_user_filter     = (&(cn=%s)(objectclass=person))
ldap_group_base_dn   = "ou=groups, dc=tibco, dc=com"
ldap_group_scope     = subtree
ldap_group_filter    = (&(cn=%s)(objectclass=groupofnames))
ldap_static_group_class    = groupofnames
ldap_static_group_attribute    = cn
ldap_static_member_attribute    = member
ldap_static_group_member_filter    =
(&(member=%s)(objectclass=groupofnames))
```

This is the basic configuration, though we will be adding additional parameters to enable SSL communications later in this document. This should seem familiar.

We have:

- The URL – LDAP is on port 389 by default
- The Principal – the fully qualified Distinguished Name for root
- The Credential – this is the EMS Term for rootpw – the password; note that EMS doesn't support any of the Hashing/Crypto – mentioned earlier - as of version 4.2.
- The User_Class – what class is going to be used for the ID?
- The User_Attribute – which field will we use? Often you may want a field for UID, but here we use the Common Name
- The User_Base_DN – where to start looking

See Chapter 9 of the EMS Users' Guide for more details.

If you don't want to use the LDAP Administrator ID and Password, you need to make sure that the ID has "read" permissions for "*". Given the structure we have built, you also need to change the DN for `ldap_principal` to include the OU.

```
ldap_principal      = "cn=test,ou=people,dc=tibco,dc=com"
ldap_credential     = test
```

Debugging EMS

You can start EMS with a trace flag:

```
tibemsd -config tibemsd.conf -trace LDAP_DEBUG
```

Testing Authentication

Java Programs

TIBCO Enterprise Message Service comes with a variety of test programs – so let's see how this works with the trace flag set! Here I will use the `tibjmsTopicPublisher` program with user `test` and password `test` -

```
C:\tibco\ems\samples\java>java tibjmsTopicPublisher -topic test.topic -user
test -password test "Hi"
```

```
-----
tibjmsTopicPublisher SAMPLE
-----
Server..... localhost
User..... test
Topic..... test.topic
Message Text.....
      Hi
-----
```

```
Publishing on topic 'test.topic'
```

```
Published message: Hi
```

```
C:\tibco\ems\samples\java>
```

```
Everything worked!
```

Here is what you get when you pass the wrong ID/Password:

```
-----
tibjmsTopicPublisher SAMPLE
-----
```

```
Server..... localhost
User..... test
Topic..... test.topic
Message Text.....
Hi
-----
```

Publishing on topic 'test.topic'

```
javax.jms.JMSSecurityException: invalid name or password
    at com.tibco.tibjms.Tibjmsx.buildException(Tibjmsx.java:549)
    at com.tibco.tibjms.TibjmsConnection._create(TibjmsConnection.java:693)
    at com.tibco.tibjms.TibjmsConnection.<init>(TibjmsConnection.java:1932)
    at
com.tibco.tibjms.TibjmsTopicConnection.<init>(TibjmsTopicConnection.java:37)
    at com.tibco.tibjms.TibjmsxCFImpl._createImpl(TibjmsxCFImpl.java:139)
    at
com.tibco.tibjms.TibjmsxCFImpl._createConnection(TibjmsxCFImpl.java:201)
    at
com.tibco.tibjms.TibjmsTopicConnectionFactory.createTopicConnection(TibjmsTopic
ConnectionFactory.java:84)
    at tibjmsTopicPublisher.<init>(tibjmsTopicPublisher.java:113)
    at tibjmsTopicPublisher.main(tibjmsTopicPublisher.java:145)
```

The partial output of the successful test with the trace flag is shown below:

```

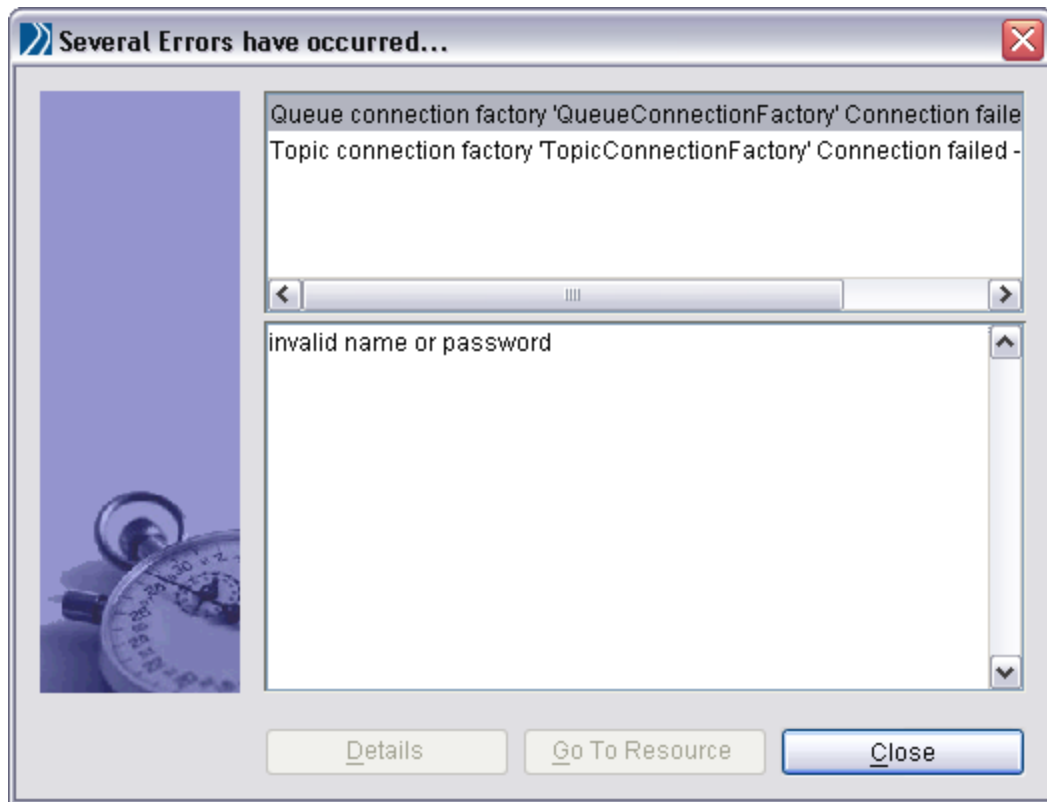
C:\> Command Prompt - startTOE-noSSL
2005-08-24 14:19:45 Server name: 'EMS-SERVER'.
2005-08-24 14:19:45 Storage Location: 'datastore'.
2005-08-24 14:19:45 Routing is disabled.
2005-08-24 14:19:45 Authorization is enabled.
2005-08-24 14:19:45 Secure Socket Layer is enabled, using OpenSSL 0.9.7e.
2005-08-24 14:19:45 Accepting connections on tcp://CMILONO-NB:7222.
2005-08-24 14:19:45 Accepting connections on ssl://CMILONO-NB:7243.
2005-08-24 14:19:45 Recovering state, please wait.
2005-08-24 14:19:45 Server is active.
2005-08-24 14:23:28 ldap_simple_bind_s("cn=root,dc=tibco,dc=com", *****)
2005-08-24 14:23:28 ldap_search_s(ab96f0, "ou=people,dc=tibco,dc=com", LDAP_SCOPE_SUBTREE, "<&(cn=test)<objectclass=person>)"
l)
2005-08-24 14:23:29 LDAP response resulting from checking existence:
2005-08-24 14:23:29 dn: cn=test,ou=people,dc=tibco,dc=com
2005-08-24 14:23:29 l: Palo alto
2005-08-24 14:23:29 cn: test
2005-08-24 14:23:29 sn: test
2005-08-24 14:23:29 mail: test@tibco.com
2005-08-24 14:23:29 givenName: test
2005-08-24 14:23:29 postalAddress: 3307 Hillview Avenue
2005-08-24 14:23:29 st: CA
2005-08-24 14:23:29 homePhone: 650-846-5559
2005-08-24 14:23:29 objectClass: top
2005-08-24 14:23:29 objectClass: person
2005-08-24 14:23:29 objectClass: organizationalPerson
2005-08-24 14:23:29 objectClass: inetOrgPerson
2005-08-24 14:23:29 telephoneNumber: 650-846-5555
2005-08-24 14:23:29 facsimileTelephoneNumber: 650-846-5556
2005-08-24 14:23:29 userPassword: test
2005-08-24 14:23:29 postalCode: 94304
2005-08-24 14:23:29 ou: people
2005-08-24 14:23:29 pager: 650-846-5555
2005-08-24 14:23:29 mobile: 650-846-5558
2005-08-24 14:23:29 physicalDeliveryOfficeName: TIBCO Software Inc.
2005-08-24 14:23:29 ldap_simple_bind_s("cn=test,ou=people,dc=tibco,dc=com", *****)
2005-08-24 14:23:29 ldap_simple_bind_s("cn=root,dc=tibco,dc=com", *****)
2005-08-24 14:23:29 ldap_search_s(ab96f0, "ou=groups, dc=tibco, dc=com", LDAP_SCOPE_SUBTREE, "<&(member=cn=test,ou=people,dc=
c=com)<objectclass=groupofnames>)", [cn, member, NULL])
2005-08-24 14:23:29 Results of searching for static groups:
2005-08-24 14:23:29 dn: cn=sales,ou=groups,dc=tibco,dc=com
2005-08-24 14:23:29 cn: sales
2005-08-24 14:23:29 member: cn=test,ou=people,dc=tibco,dc=com
2005-08-24 14:23:29 dn: cn=admin-west,ou=groups,dc=tibco,dc=com
2005-08-24 14:23:29 cn: admin-west
2005-08-24 14:23:29 member: cn=test,ou=people,dc=tibco,dc=com
2005-08-24 14:23:29 ldap_search_s(ab96f0, "ou=groups, dc=tibco, dc=com", LDAP_SCOPE_SUBTREE, "<&(cn=sales)<objectclass=groupo
>)", [NULL])
2005-08-24 14:23:29 LDAP response resulting from getting attributes for group 'sales':
2005-08-24 14:23:29 dn: cn=sales,ou=groups,dc=tibco,dc=com
2005-08-24 14:23:29 objectClass: top
2005-08-24 14:23:29 objectClass: groupOfNames
2005-08-24 14:23:29 cn: sales
2005-08-24 14:23:29 ou: groups
2005-08-24 14:23:29 member: cn=test,ou=people,dc=tibco,dc=com
2005-08-24 14:23:29 ldap_search_s(ab96f0, "ou=groups, dc=tibco, dc=com", LDAP_SCOPE_SUBTREE, "<&(member=cn=sales,ou=groups, d
, dc=com)<objectclass=groupofnames>)", [cn, member, NULL])
2005-08-24 14:23:29 Results of searching for static groups:

```

You can see that you get information about the binding, search, scope, and the results.

BusinessWorks

A failure in configuring BusinessWorks is shown – no additional information is given in the Console Window for version 5.2.X:



LDAP/S – securing the connection between EMS and LDAP

There are two modes for creating a secure connection between an LDAP Client and the Server – one is LDAP/S which is very much like HTTP/S in that it has a well-known secure port – port 636. The other mode is one where the client negotiates its desire for SSL on the standard port – port 389 – and it is called “startTLS”. We will cover the first case, for the sake of brevity.

As you might imagine, we will need to do the following:

- assign X.509v3 Certificates to both OpenLDAP and EMS
- point to the appropriate self-signed Certificate for the Certificate Authority
- edit the configuration files

All the acronyms used in LDAP will be found in the certificate – LDAP came from the X.500 directory project and...it is no coincidence that X.509 certificates have the same origin!

Create or find appropriate X.509v3 Certificates

The “Subject’s DN” needs to be fully qualified for LDAP. I used OpenSSL to create a self-signed CA Certificate which I then used to sign two other certificates - in PEM Format. For the Common Name, I used my machine-name.domain (cmilono-nb.na.tibco.com) for both the Client and Server Certificates. So, I now have the following:

```
C:\Certificates>dir
Volume in drive C has no label.
Volume Serial Number is 2C53-E420

Directory of C:\Certificates

08/15/2005  05:24 PM    <DIR>          .
08/15/2005  05:24 PM    <DIR>          ..
08/15/2005  04:47 PM                984 cacert.der
08/15/2005  05:08 PM            1,391 cacert.pem
08/15/2005  05:08 PM                951 cakey.pem
08/24/2005  09:17 AM            3,792 Client-cert.pem
08/24/2005  09:17 AM                963 Client-key.pem
08/24/2005  09:14 AM            3,806 Server-cert.pem
08/24/2005  09:14 AM                963 Server-Key.pem
```

CACERT.DER is used by OpenSSL – all the others are used by OpenLDAP and EMS.

Re-edit slapd.conf

We simply add a few more configuration parameters, mainly pointing to the certificates (additions shown in bold):

```
#
# See slapd.conf(5) for details on configuration options.
# This file should NOT be world readable.
# This is the default
ucdata-path      C:/openldap/ucdata
include          C:/openldap/etc/schema/core.schema

# Add this as being very useful - Carlo Milono
include          C:/openldap/etc/schema/cosine.schema
include          C:/openldap/etc/schema/inetorgperson.schema
include          C:/openldap/etc/schema/java.schema
include          C:/openldap/etc/schema/openldap.schema

TLSCACertificateFile      C:/Certificates/cacert.pem
#TLSCACertificatePath    C:/Certificates
TLSCertificateFile       C:/Certificates/Server-cert.pem
TLSCertificateKeyFile    C:/Certificates/Server-key.pem
TLSCipherSuite           HIGH:MEDIUM
#TLSRandFile
```

Re-edit tibemsd.conf

The only changes will be to modify the URL to point to ldaps: and the famous port 636, and then the type of connection (ldaps), and to point to the certificates, using the “client” certs instead of the “server” certs (changes and additions shown in bold):

```

ldap_url           = ldaps://localhost:636
ldap_principal      = "cn=root,dc=tibco,dc=com"
ldap_credential     = secret
ldap_user_class     = person
ldap_user_attribute = cn
ldap_user_base_dn   = "ou=people,dc=tibco,dc=com"
ldap_user_scope     = subtree
ldap_user_filter    = (&(cn=%s)(objectclass=person))
ldap_group_base_dn  = "ou=groups,dc=tibco,dc=com"
ldap_group_scope    = subtree
ldap_group_filter   = (&(cn=%s)(objectclass=groupofnames))
ldap_static_group_class = groupofnames
ldap_static_group_attribute = cn
ldap_static_member_attribute = member
ldap_static_group_member_filter = (&(member=%s)(objectclass=groupofnames))

ldap_conn_type=ldaps
ldap_tls_cacert_file=C:\Certificates\cacert.pem
ldap_tls_cert_file=C:\Certificates\client-cert.pem
ldap_tls_key_file=C:\Certificates\client-key.pem

```

New Command-line for Starting OpenLDAP

You need to specify the new URL, and probably want to have finer-grained debugging information for OpenLDAP, so you can use the following command-line:

```
slapd -d 127 -h ldaps://localhost ldap://localhost
```

New Procedures for Start-Up

In both the OpenLDAP and EMS, you will have to provide the Password/Key to the X.509v3 certificates being used:

```
...
2005-08-24 15:22:19 Detected IP interface: 10.105.150.65 (IP04)
2005-08-24 15:22:19 Detected IP interface: 192.168.0.3 (IP05)
2005-08-24 15:22:19 Detected IP interface: 127.0.0.1 (loopback)
2005-08-24 15:22:19 Reading configuration from 'tibemsd-TOE.conf'.
2005-08-24 15:22:19 Logging into file 'C:\TIBCO\EMS\BIN\LOGFILE.TXT'
2005-08-24 15:22:19 Server name: 'EMS-SERVER'.
2005-08-24 15:22:19 Storage Location: 'datastore'.
2005-08-24 15:22:19 Routing is disabled.
2005-08-24 15:22:19 Authorization is enabled.
2005-08-24 15:22:19 Secure Socket Layer is enabled, using OpenSSL 0.9.7e.
2005-08-24 15:22:20 Accepting connections on tcp://CMILON0-NB:7222.
2005-08-24 15:22:20 Accepting connections on ssl://CMILON0-NB:7243.
2005-08-24 15:22:20 Recovering state, please wait.
2005-08-24 15:22:20 Server is active.
Enter SSL password:
```

This prompt usually comes after the first JMS Message which initiates the SSL connection.

For OpenLDAP – you are prompted at the end of initialization:

```
...
distinguishedName $ aliasedObj
gContexts $ subschemaSubentry $ modifiersName $ creatorsName ) )
2.5.13.0 (objectIdentifierMatch): matchingRuleUse: ( 2.5.13.0 NAME
'objectIdentifierMatch' APPLIES ( supportedApp
$ supportedFeatures $ supportedExtension $ supportedControl ) )
Enter PEM pass phrase:
```

Listening on the Port for LDAP/S - 636

```
netstat -a | grep ldap
TCP        CMILON0-NB:ldaps          CMILON0-NB.na.tibco.com:0 LISTENING
TCP        CMILON0-NB:ldaps          localhost:2640          ESTABLISHED
TCP        CMILON0-NB:ldaps          localhost:2642          ESTABLISHED
TCP        CMILON0-NB:2640           localhost:ldaps         ESTABLISHED
TCP        CMILON0-NB:2642           localhost:ldaps         ESTABLISHED
```

Testing Authorization

In this section, we will do the following:

- Configure a queue
- Grant permissions on that queue to the group “sales” (which has user “test”)
- Perform a test that should fail
- Realize we needed to put the ‘secure’ mode onto the queue
- Re-test

Create Queue

Start up tibemsadmin and create a queue named “sales.send”.

create queue sales.send

Grant Partial Permissions

grant queue sales.send group=sales receive, browse

With this permission, user “test” should *not* be able to send to queue “sales.send”, but we have forgotten something.....

First Test

```
C:\tibco\ems\samples\java>java tibjmsQueueSender -queue sales.send -user test -password test "Hi"
```

```
-----
tibjmsQueueSender SAMPLE
-----
```

```
Server..... localhost
User..... test
Queue..... sales.send
-----
```

```
Publishing into queue: 'sales.send'
```

```
Sent message: Hi
```

We forgot the SECURE Property!

Using the tibemsadmin program, change the property for the queue as follows:

addprop queue sales.send secure

This will enforce the permissions, so our test should fail this time, and it does!

```
C:\tibco\ems\samples\java>java tibjmsQueueSender -queue sales.send -user test -password test "Hi"
```

```
-----
tibjmsQueueSender SAMPLE
-----
```

```
Server..... localhost
User..... test
Queue..... sales.send
-----
```

```
Publishing into queue: 'sales.send'
```

```
javax.jms.JMSSecurityException: Not permitted
    at com.tibco.tibjms.Tibjmsx.buildException(Tibjmsx.java:549)
    at com.tibco.tibjms.TibjmsSession._createProducer(TibjmsSession.java:714)
    at com.tibco.tibjms.TibjmsQueueSession.createSender(TibjmsQueueSession.java:123)
    at tibjmsQueueSender.<init>(tibjmsQueueSender.java:113)
    at tibjmsQueueSender.main(tibjmsQueueSender.java:136)
```

You can change the permission on the queue – the queue is named sales.send, so our user should be able to send!

Enter this command in tibemsadmin:

grant queue sales.send group=sales send, receive, browse

The output of the EMS Console will show the following:

```
2005-08-25 10:49:22 [test@CMILONO-NB]: create sender failed: access denied for queue [sales.send].
2005-08-25 10:50:21 [admin@CMILONO-NB]: granted group 'sales' permissions on queue 'sales.send':
receive, send, browse
```

Modification – adding UID & using UID rather than CN

Most Enterprise Applications give you a User Identifier (User ID), and the core.schema used by LDAP includes such an element:

```
# Derived from RFC 1274, but with new "short names"
#
attributetype ( 0.9.2342.19200300.100.1.1
                NAME ( 'uid' 'userid' )
                DESC 'RFC1274: user identifier'
                EQUALITY caseIgnoreMatch
                SUBSTR caseIgnoreSubstringsMatch
                SYNTAX 1.3.6.1.4.1.1466.115.121.1.15{256} )
```

To be able to use this in LDAP, we simply need to add the element to relevant records as the schema is already available.

addUID.ldif

Here is a partial LDIF file with changetype equal to modify:

```
## Add the core.schema "uid" as a field for existing users
dn: cn=test, ou=people, dc=tibco, dc=com
changetype: modify
add: uid
uid: t

#dn: cn=north, ou=people, dc=tibco, dc=com
#changetype: modify
#add: uid
#uid: n
```

Note that you need to have a blank line to separate each entry.

LDAPModify Command

The following command will modify entries in LDAP:

```
ldapmodify -D "cn=root,dc=tibco,dc=com" -w secret -x -v -f addUID.ldif
```

Modify tibemsd.conf to use the UID attribute

You need to change the `ldap_user_attribute` from `cn` to `uid` and make the same change for `ldap_user_filter` – changes shown in bold below:

```
ldap_url          = ldap://localhost:389
ldap_principal    = "cn=root,dc=tibco,dc=com"
ldap_credential   = secret
ldap_user_class   = person
ldap_user_attribute = uid
ldap_user_base_dn = "ou=people,dc=tibco,dc=com"
ldap_user_scope   = subtree
ldap_user_filter = (&(uid=%s)(objectclass=person))
ldap_group_base_dn = "ou=groups, dc=tibco, dc=com"
ldap_group_scope  = subtree
ldap_group_filter = (&(cn=%s)(objectclass=groupofnames))
ldap_static_group_class = groupofnames
ldap_static_group_attribute = cn
ldap_static_member_attribute = member
ldap_static_group_member_filter =
(&(member=%s)(objectclass=groupofnames))
```